

Báo cáo tiến độ tuần từ ngày 23/3 đến ngày 28/3

Tìm hiểu mạng Lora và thiết kế gói tin

Phần 1: Tổng quan về LoRa và Cách sử dụng

LoRa (Long Range) là một công nghệ truyền thông không dây tầm xa, tiêu thụ điện năng thấp. **LoRa P2P (Point-to-Point)** hoặc mạng hình sao tự xây dựng (Star topology: nhiều Node gửi đến một Gateway).

Để sử dụng LoRa hiệu quả, bạn cần nắm vững **4 thông số cốt lõi** sau (thường được cấu hình qua thư viện như `LoRa.h` của Sandeep Mistry hoặc `RadioLib` trên ESP32):

1. **Tần số (Frequency):** Module SX1278 hoạt động ở dải **433 MHz** (ở Việt Nam và châu Á, dải này hợp lệ cho thiết bị công suất thấp). Các module muốn giao tiếp với nhau phải set chung một tần số.
2. **Spreading Factor (SF - Hệ số lan truyền):** Từ SF7 đến SF12.
 - **SF thấp (SF7, SF8):** Tốc độ truyền nhanh, thời gian phát ngắn (ít tốn pin), nhưng khoảng cách ngắn. (Dùng cho trong nhà, khoảng cách < 1km).
 - **SF cao (SF11, SF12):** Tốc độ truyền rất chậm, tốn pin hơn nhưng có thể truyền cực xa (phá xuyên tường tốt, có thể lên đến 5-10km).
3. **Bandwidth (BW - Băng thông):** Thường để **125 kHz**. Băng thông rộng hơn (250kHz) thì truyền nhanh hơn nhưng độ nhạy thu giảm.
4. **Sync Word:** Một mã định danh (từ 0x00 đến 0xFF) để Gateway và Node phân biệt mạng của bạn với các mạng LoRa khác xung quanh. (Cả Node và Gateway phải chung Sync Word).

Phần 2: Nguyên tắc Thiết kế gói tin (Packet Design)

Với Wifi hay MQTT, ta có thể thoải mái dùng chuỗi **JSON** (ví dụ: `{"node":1, "temp":25.5, "pm25":12}`). Nhưng với **LoRa** thì **TUYỆT ĐỐI KHÔNG NÊN** dùng **JSON** hay chuỗi `String` dài.

Lý do:

- LoRa có tốc độ data rate rất thấp (chỉ khoảng vài chục đến vài trăm byte mỗi giây).
- Gói tin càng dài (Time-on-Air lớn): Càng tốn pin, càng dễ bị nhiễu làm hỏng gói tin, và vi phạm quy định về thời gian phát sóng (Duty Cycle).
- Payload tối đa của SX1278 chỉ là **255 bytes**.

Giải pháp: đóng gói dữ liệu thành các dạng cơ bản (byte/nhi phân) bằng cách dùng `struct` (Cấu trúc) trong C/C++.

Phần 3: thiết kế gói tin thực tế cho dự án

Bước 1: Định nghĩa cấu trúc gói tin (Struct)

Thay vì gửi số float (4 bytes) cho nhiệt độ, ta nhân với 10 và gửi số nguyên (2 bytes) để tiết kiệm. Ví dụ: `25.5°C` → gán thành `255`. Lên Gateway chia lại cho 10.

[LoraPacket.drawio - draw.io](https://lora.packet.drawio.com)

- **Node ID (1 byte):** Định danh node (0x01 → 0xFF, tối đa 255 node).
- **Pkt Type (1 byte):** Loại gói tin (Ví dụ: `0x01` là Data đọc từ cảm biến, `0x02` là Heartbeat cảnh báo, v.v.).
- **Msg ID (1 byte):** Bộ đếm gói tin (0-255), tự động tăng sau mỗi lần gửi để phát hiện nếu có gói tin bị rớt/mất trên đường truyền.
- **PM1.0/2.5/PM10 (6 bytes - mỗi loại 2 bytes):** Giá trị bụi mịn ($\mu\text{g}/\text{m}^3 \times 10$) để giữ lại 1 chữ số thập phân.
- **CO2 (2 bytes):** Nồng độ khí CO₂ (ppm), có dải đo từ 400 đến 8192.
- **TVOC (2 bytes):** Tổng lượng chất hữu cơ dễ bay hơi trong không khí (ppb), dải đo từ 0 đến 1187.
- **Temp (2 bytes):** Nhiệt độ môi trường ($^{\circ}\text{C} \times 10$) để giữ 1 chữ số thập phân (có thể là số âm).
- **Humidity (2 bytes):** Độ ẩm môi trường ($\% \times 10$) để giữ 1 chữ số thập phân.
- **Battery (1 byte):** Trạng thái mức pin còn lại của Sensor Node, định dạng từ 0-100%.

Bước 1: Khai báo Struct (Dùng chung cho cả Node và Gateway)

```

cpp
// Thêm __attribute__((packed)) để báo compiler không tự động thêm thẻ padding
// giúp tối ưu tuyệt đối kích thước.
typedef struct __attribute__((packed)) {
uint8_t  nodeId;           // 1 byte - Định danh node (1-255)
uint8_t  pktType;         // 1 byte - Loại gói tin (Ví dụ: 0x01 là Data)
uint8_t  msgId;           // 1 byte - Số thứ tự gói tin để check rớt mạng (0-255)
uint16_t pm1;
uint16_t pm25;            // 2 bytes - Bụi PM2.5 ( $\mu\text{g}/\text{m}^3 \times 10$ )
uint16_t pm10;            // 2 bytes - Bụi PM10 ( $\mu\text{g}/\text{m}^3 \times 10$ )
uint16_t co2;             // 2 bytes - Nồng độ CO2 (ppm)
uint16_t tvoc;            // 2 bytes - Nồng độ TVOC (ppb)
int16_t  temperature;     // 2 bytes - Nhiệt độ ( $^{\circ}\text{C} \times 10$ )
uint16_t humidity;        // 2 bytes - Độ ẩm ( $\% \times 10$ )
uint8_t  battery;         // 1 byte - Phần trăm pin (0-100%)
} SensorPayload;

```

Bước 2: Code tại Sensor Node (TX - Gửi dữ liệu)

```

cpp
#include<LoRa.h>

SensorPayload myData;
uint8_t messageCount =0; // Bộ đếm Msg ID (0-255)

void sendLoRaData() {
    // 1. Cập nhật Header vào struct
    myData.nodeId =1;           // Định danh Node 1
    myData.pktType =0x01;       // Gói Data (0x01)
    myData.msgId = messageCount++; // Lưu bộ đếm hiện tại rồi tăng lên 1

    // 2. Đọc cảm biến (Ví dụ)
    float pm25_val = pms.getPM25(); // Giả sử 12.5
}

```

```

float pm10_val = pms.getPM10(); // Giả sử 15.0
float t = dht.readTemperature(); // Giả sử 25.4
float h = dht.readHumidity();    // Giả sử 60.5

// 3. Nạp dữ liệu vào struct (Nhân 10 để ép về số nguyên)
myData.pm25 = (uint16_t)(pm25_val *10.0); // Lưu 125
myData.pm10 = (uint16_t)(pm10_val *10.0); // Lưu 150
myData.co2 = ccs.geteCO2();                // Lưu 450
myData.tvoc = ccs.getTVOC();                // Lưu 120
myData.temperature = (int16_t)(t *10.0);   // Lưu 254
myData.humidity = (uint16_t)(h *10.0);     // Lưu 605
myData.battery = getBatteryPercentage();   // Lưu 85

// 4. Gửi đi qua LoRa (Truyền nguyên khối memory bao gồm
16 bytes)
LoRa.beginPacket();
LoRa.write((uint8_t *)&myData, sizeof(SensorPayload));
LoRa.endPacket();

Serial.printf("Đã gửi gói tin LoRa: %d bytes\\n", sizeof(S
ensorPayload));
}

```

Bước 3: Code tại Gateway (RX - Nhận dữ liệu)

```

cpp
#include<LoRa.h>

SensorPayload receivedData;

void onReceive(int packetSize) {
    // Kiểm tra xem dung lượng gói tin tới có đúng 16 byte củ
a struct không
    if (packetSize !=sizeof(SensorPayload)) {
        Serial.println("Lỗi: Kích thước gói tin không đúng (Bỏ
qua)");
    }
    return;
}

```

```

}

// Đọc nguyên chuỗi byte đảy thẳng vào memory của struct
LoRa.readBytes((uint8_t *)&receivedData, packetSize);

// Lấy các thông số độ mạnh sóng của LoRa
int rssi = LoRa.packetRssi();
float snr = LoRa.packetSnr();

// Khôi phục lại dữ liệu ban đầu (Chia 10)
float realPm25 = receivedData.pm25 /10.0;
float realPm10 = receivedData.pm10 /10.0;
float realTemp = receivedData.temperature /10.0;
float realHum = receivedData.humidity /10.0;

// In ra màn hình Serial để debug kiểm tra
Serial.println("-----");
");
Serial.printf("Nhận từ Node %d | MsgID: %d | PktType: 0x%
02X | RSSI: %d dBm | SNR: %.2f\\n",
                receivedData.nodeId, receivedData.msgId, r
eceivedData.pktType, rssi, snr);

Serial.printf("PM2.5: %.1f µg/m³ | PM10: %.1f µg/m³\\n",
realPm25, realPm10);
Serial.printf("CO2: %d ppm | TVOC: %d ppb\\n", receivedDa
ta.co2, receivedData.tvoc);
Serial.printf("Nhiệt độ: %.1f C | Độ ẩm: %.1f %%\\n", rea
lTemp, realHum);
Serial.printf("Pin Node: %d%%\\n", receivedData.battery);
Serial.println("-----");
");

// Xử lý tạo chuỗi JSON để đảy lên Web (VD Node.js backen
d)
}

```

Cơ sở dữ liệu với postgresSQL

1. Lý do lựa chọn PostgreSQL + Extensions

Hệ thống IoT giám sát chất lượng không khí của dự án (theo [Các chức năng của hệ thống \(dự kiến\).md](#)) có 3 loại dữ liệu đặc thù kết hợp. Việc sử dụng PostgreSQL và 2 extension giúp giải quyết toàn diện chỉ trong MỘT hệ quản trị CSDL duy nhất:

A. Dữ liệu thời gian thực (Time-series) -> TimescaleDB

- Dữ liệu từ các sensor node (PM2.5, PM10, CO2, Nhiệt độ, TVOC...) gửi về liên tục thông qua Gateway sẽ tạo ra một lượng dữ liệu chuỗi thời gian khổng lồ.
- TimescaleDB** biến bảng PostgreSQL thông thường thành các **Hypertable**, tự động phân mảnh (partition) khối lượng dữ liệu theo trục thời gian, giúp tốc độ ghi (INSERT) và truy xuất (SELECT) cực kỳ nhanh.
- Continuous Aggregates**: Tự động tính toán trước các giá trị (ví dụ: trung bình PM2.5 mỗi giờ, mỗi ngày). Giúp chức năng "Xem lịch sử theo Giờ / Ngày" trên Dashboard load biểu đồ gần như ngay lập tức mà Backend không cần tốn dụng lượng RAM tính toán Group By.

B. Dữ liệu định vị (Geospatial) -> PostGIS

- Vị trí lắp đặt của các Sensor Node và Gateway là điểm trên bản đồ thực tế (Kinh độ / Vĩ độ).
- PostGIS** hỗ trợ chuẩn hóa kiểu lưu trữ hình học (ví dụ: **GEOMETRY(Point)**) và các thuật toán tìm kiếm không gian.
- Phục vụ trực tiếp chức năng: *Dựa vào vị trí người dùng để gợi ý trạm đo gần nhất và Bản đồ AQI toàn khu vực.*

C. Dữ liệu quan hệ (Relational Data) -> PostgreSQL Lõi

- Quản lý người dùng, phân quyền Admin, trạng thái/cấu hình của thiết bị, bảng log hệ thống các chỉnh sửa từ Admin.
- Tính toàn vẹn dữ liệu (ACID) của PostgreSQL đảm bảo độ chính xác cho các logic này.

2. Thiết kế Schema dự kiến

2.1. Các bảng Dữ liệu quan hệ (Relational Tables)

Bảng `users` (Người dùng & Quản trị viên)

- `id` (UUID, Primary Key)
- `username` (VARCHAR)
- `password_hash` (VARCHAR)
- `role` (ENUM: 'admin', 'user')
- `created_at` (TIMESTAMPTZ)

Bảng `gateways` (Trạm trung chuyển LoRa-WiFi)

- `id` (VARCHAR, Primary Key - Mac Address của thiết bị)
- `name` (VARCHAR)
- `location_desc` (VARCHAR)
- `status` (ENUM: 'online', 'offline')
- `last_seen` (TIMESTAMPTZ) - Phục vụ chức năng giám sát Last seen từ Admin

Bảng `sensor_nodes` (Trạm cảm biến - Tích hợp PostGIS)

- `id` (VARCHAR, Primary Key)
- `gateway_id` (Foreign Key → gateways.id)
- `name` (VARCHAR)
- `geom` (GEOMETRY(Point, 4326)) - Lưu tọa độ kinh tuyến, vĩ tuyến bằng PostGIS
- `status` (ENUM: 'active', 'inactive', 'lost_connection')
- `battery_level` (INT)
- `lora_rssi` (INT) - Mức độ tín hiệu kết nối LoRa

2.2. Bảng Dữ liệu Chuỗi thời gian (Hypertable - TimescaleDB)

Bảng `measurements` (Chỉ số đo lường Sensor)

- `time` (TIMESTAMPTZ, Index quan trọng nhất)
- `node_id` (VARCHAR, Foreign Key → sensor_nodes.id)
- `pm25` (FLOAT)

- `pm10` (FLOAT)
- `co2` (INT)
- `tvoc` (INT)
- `temperature` (FLOAT)
- `humidity` (FLOAT)

Câu lệnh SQL để cấu hình Hypertable:

```
SELECT create_hypertable('measurements', 'time');
```

3. Ứng dụng triển khai thuật toán tính năng

3.1. Tìm trạm đo gần người dùng nhất (Ứng dụng PostGIS)

Khi người dùng truy cập Web/App từ điện thoại và gửi tọa độ (Kinh độ `lng`, Vĩ độ `lat`), Backend dùng 1 câu lệnh SQL duy nhất để tìm trạm gần nhất thay vì phải dùng công thức Haversine để tính khoảng cách với tất cả các trạm:

```
SELECT id, name, location_desc,
       ST_Distance(geom, ST_SetSRID(ST_MakePoint(lng, lat),
4326), true) as distance_meters
FROM sensor_nodes
ORDER BY geom <-> ST_SetSRID(ST_MakePoint(lng, lat), 4326)
LIMIT 1;
```

3.2. Dashboard thời gian thực và Biểu đồ Lịch sử (Ứng dụng TimescaleDB)

Thay vì SQL truyền thống phải chạy lệnh SUM/AVG cho vài triệu raw data record mỗi khi User/Admin bấm "Xem lịch sử", DBA sẽ cấu hình một view chạy ngầm (Continuous Aggregates):

```
CREATE MATERIALIZED VIEW hourly_measurements
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', time) AS bucket_time,
       node_id,
       AVG(pm25) as avg_pm25,
```



```
    AVG(pm10) as avg_pm10,  
    AVG(co2) as avg_co2,  
    MAX(temperature) as max_temp  
FROM measurements  
GROUP BY bucket_time, node_id;
```

Khi Dashboard gọi API lịch sử ngày, API chỉ gọi `SELECT * FROM hourly_measurements`, tốc độ phản hồi chỉ vài mili-giây.

3.3. Tự động dọn dẹp dữ liệu (Data Retention)

Sensor gửi 5-10 phút/lần sẽ gây đầy Local Storage trên hệ thống Server. Giải pháp ứng dụng là thiết lập chức năng Retention của TimescaleDB xóa dữ liệu raw cũ quá 3 tháng, nhưng **vẫn giữ lại** dữ liệu đã nhóm (Continuous Aggregates) để làm báo cáo thống kê các năm.

```
SELECT add_retention_policy('measurements', INTERVAL '3 mon  
ths');
```

Thiết kế server

1. Tổng quan Công nghệ (Tech Stack)

- **Framework Core:** Node.js với **ExpressJS** (như trong bản vẽ hệ thống). Theo định hướng I/O non-blocking, cực kỳ phù hợp để xử lý hàng ngàn HTTP request liên tục từ các Gateway.
- **Database:**
 - Nên sử dụng **PostgreSQL** kết hợp với extension **TimescaleDB**. TimescaleDB đặc biệt tối ưu cho dữ liệu chuỗi thời gian (time-series) của cảm biến, cung cấp các hàm tổng hợp theo khung thời gian (giờ/ngày) cực nhanh.
 - Sử dụng **PostGIS** để xử lý truy vấn không gian (spatial query) cho chức năng tìm trạm gần nhất theo vị trí GPS.
- **Real-time (Tùy chọn):** Dùng **Socket.io** để đẩy dữ liệu cảm biến mới nhất và trạng thái thiết bị lên Dashboard React theo thời gian thực (thay vì polling).

2. Cấu trúc thư mục Server (Folder Structure)

thiết kế server theo mô hình **MVC** (hoặc Module-based) kết hợp **Service-Layer/Controller-Layer** để tách bạch logic nghiệp vụ với routing:

```
text
backend/
├─ src/
│   ├─ controllers/      # Xu'ly Request/Response cho từng
│   │                   module (User, Admin, Ingestion...)
│   ├─ services/         # Logic tính toán phức tạp (Tính
│   │                   AQI, lọc theo vị trí, xuất CSV...)
│   ├─ models/           # Định nghĩa cấu trúc Database (S
│   │                   equelize/Prisma hoặc raw query)
│   ├─ routes/           # Định nghĩa các endpoint (User A
│   │                   PI, Admin API, Device API)
│   ├─ middlewares/      # Interceptor (Auth JWT, Roles, R
│   │                   ate Limiting, Validation)
│   ├─ utils/            # Các helper (Logger, tính khoảng
│   │                   cách GPS, mã hóa, format ngày)
│   └─ server.js         # File khởi tạo Express, cấm midd
│   │                   leware, kết nối DB
│   └─ .env              # Lưu secrets, thông số cấu hình
Server
```

3. Thiết kế các cụm API (Routing Design)

Dựa trên chức năng dự kiến, Server cần thiết kế 3 nhóm API chính:

A. Ingestion API (Dành riêng cho IoT Gateway)

Chịu trách nhiệm nhận dữ liệu viễn thông từ các Gateway, yêu cầu tốc độ cao và tối ưu.

- **POST /api/v1/telemetry** : Endpoint để Gateway đẩy mảng dữ liệu qua HTTP POST.
 - *Logic*: Xác thực Secret Key của Gateway → Lưu bản ghi đo lường (PM2.5, CO2...) vào DB → Cập nhật mốc **last_seen** của Gateway & thiết bị Node → Giao cho service nền kiểm tra ngưỡng cảnh báo.

B. Public / User API (Cho ứng dụng Dashboard hiển thị)

Chịu trách nhiệm lấy dữ liệu phân phối ra front-end, tối ưu caching.

- `GET /api/v1/stations/nearest` : Gợi ý trạm gần người dùng (nhận lat, lng query).
- `GET /api/v1/stations/dashboard` : Lấy thông số đo lường mới nhất của tất cả các khu vực để hiển thị lên bản đồ (Map AQI).
- `GET /api/v1/stations/:id/history` : Xem lịch sử theo từng thông số (chế độ xem theo khoảng thời gian) phục vụ vẽ biểu đồ Bar/Line.
- `GET /api/v1/stations/ranking` : Lọc và sắp xếp các trạm theo mức độ ô nhiễm từ cao xuống thấp.
- `GET /api/v1/weather/current` : Controller đóng vai trò Proxy để gọi dự báo thời tiết từ API bên thứ 3 (như OpenWeatherMap) sau đó gộp chung với dữ liệu trạm để trả về Dashboard.

C. Admin API (Bảo vệ bằng JWT & RBAC)

Quản lý danh mục, thiết bị và trích xuất.

- **Quản lý & Monitor Thiết bị:**
 - `CRUD /api/v1/admin/gateways` & `CRUD /api/v1/admin/nodes` : Thêm/xóa/sửa trạng thái, vị trí lắp đặt và mã nhận diện mạng.
 - `GET /api/v1/admin/devices/health` : Tính toán % packet loss (tỷ lệ thành công), mức pin, chất lượng sóng LoRa (RSSI), và trạng thái Online/Offline.
- **Dữ liệu & Cấu hình:**
 - `GET /api/v1/admin/export` : Trích xuất (Query dữ liệu chuỗi thời gian) ra định dạng file `.csv` hoặc `.xlsx`.
 - `PUT /api/v1/admin/configs` : Sửa đổi các tham số nhạy cảm (Chu kỳ trích mẫu, ngưỡng cảnh báo AQI).
- **Phân quyền & Audit:**
 - `POST /api/v1/admin/users` : Tạo và phân quyền cho quản trị viên mới.
 - `GET /api/v1/admin/audit-logs` : Truy vấn lịch sử các thao tác của quản trị viên hệ thống.

4. Thiết kế các Logic nền (Background Services / Cron Jobs)

Có một số logic không thể gắn vào API request thông thường mà cần chạy ngầm bên dưới Server:

1. Health Check Service (Tính trạng thái online/offline):

- Viết 1 Cron job chạy mỗi 5 hoặc 10 phút. Quét vào Database xem các thiết bị (Gateways/Nodes) có `last_seen` vượt ngưỡng `X phút` hay không. Nếu có, chuyển trạng thái thành `Offline` và tạo một System Alert Log nhắc nhở Quản trị viên.

2. AQI Calculator Engine:

- Hàm `aqiService` dùng để nhận dữ liệu PM2.5, PM10... đo đạc được, đưa vào công thức của WHO hay EPA Mỹ để xuất ra chỉ số AQI tổng thể và text diễn giải ("*ô nhiễm gấp X lần*"). Hàm này nên được tiêm ngay tại luồng Ingestion lưu vào DB, hoặc tính on-the-fly khi có request từ Dashboard tùy chiến lược lưu trữ.